Deployment, Operations & Release Runbook

Document 06 of 07

Purpose
Give maintainers a repeatable path to prepare the environment, validate code and database prerequisites, release the app, check service health, and respond to common failures. This is a repository-evidence-based handover, not a claim that a production deployment currently exists.

Current runtime	Streamlit application, Python, Neon PostgreSQL through psycopg.
Repository	rd2080/Bulk_App_Personalised, GitHub default branch main; reviewed 21 Oct 2026.
Evidence status	README documents local launch and Streamlit Cloud secrets; GitHub has CI workflow. No deploy workflow or active hosting configuration was verified.
Title assumption	The seven-document package outline naming Document 06 was unavailable here; operations runbook is the working title.

Release posture

The available evidence supports a documented local setup, a GitHub Actions CI workflow on pull requests and pushes to main, and a Neon migration process maintained as numbered SQL files. Deployment provider, production URL, approval gates, rollback automation, backups, monitoring and on-call ownership are not established in the reviewed repository.

Do not interpret a successful CI run as proof that a Neon migration is applied or that the hosted application is healthy.

1. Environment preparation

1.1 Local development

Repository README instructs maintainers to use Python 3.9+, install requirements, configure DATABASE_URL for Neon, then start Streamlit with app.py. requirements.txt currently includes Streamlit, python-dotenv, psycopg binary, Google GenAI and OpenAI clients. Provider libraries being installed does not mean AI calls are part of the implemented workflows.

Step	Action	Check
1	Use an isolated Python environment and install requirements.txt.	Dependencies resolve; Python compatibility is confirmed against the repository and deployment runtime.
2	Set DATABASE_URL locally or through Streamlit secrets. Keep credentials out of source control.	Application can resolve the secret without printing it.
3	Launch the app from repository root using Streamlit.	Home page loads; navigation pages are available.
4	Use a non-production Neon branch for schema and write checks.	Confirm branch identity before applying migrations or sample writes.

Credential handling

- Local .env and Streamlit secrets are ignored by .gitignore. The .env.example shows DATABASE_URL and future provider-key placeholders; copy values into a private local configuration, never commit real secrets.
- The connection helper checks DATABASE_URL in the environment, then Streamlit secrets, and raises a clear error when absent. Avoid logging or echoing full connection strings in support traces.
- Rotate any credential accidentally committed or exposed. This is a general operational action; no secret incident was found in the reviewed material.

1.2 Hosted configuration (provider dependent)

README points to Streamlit Cloud app settings/secrets for hosted DATABASE_URL. The actual deployed app, service account, URL and environment settings were not available to inspect. Confirm the hosting provider and environment separation before using this checklist.

- Configure the intended Neon connection string in the hosting platform’s protected secret store. Verify that it targets the intended branch/database.
- Confirm the host supports the project’s Python and package versions and that psycopg binary dependencies install successfully.
- For browser microphone access on remote hosting, use HTTPS; manual text input remains the fallback when browser speech is unavailable.

2. CI and pre-release verification

The current repository contains .github/workflows/ci.yml. It runs on pull requests and pushes to main, provisions Python 3.11, installs requirements plus pytest, and runs pytest -q. This is source-level workflow evidence only; this task did not execute CI or inspect recent workflow results.

- Require the CI workflow to pass for the candidate commit; investigate failures rather than relying on a prior run from another SHA.
- Review changed migrations and repository code together. Check that application validation and SQL constraints agree and that new migrations are safe for the target schema.
- Confirm whether a deployment workflow is added later. The attempted lookup of .github/workflows/deploy.yml returned not found; this does not exclude manual or provider-managed deployment.

Release checklist

Gate	Evidence to record
Source	Commit SHA, branch, change summary, reviewed diff.
CI	Workflow run URL/status for the same SHA.
Database	Target Neon branch/database and migration versions applied; never store secret value in release notes.
App	Hosting target, configuration change, release time and operator.
Smoke check	Home page plus critical page reads/writes; record any environment-specific limitation.

3. Database migration procedure

Numbered SQL migrations are the schema source of truth. Apply the required migration to the intended Neon branch before releasing application code that depends on it. Migration files in GitHub do not apply themselves. Current documented sequence ends at 008: weight entries, daily check-ins, food inventory, check-in weight, weight precision/range, profile, recipe library, and daily meal plans.

- Before applying: identify target branch/database; inspect current schema and applied migration history; confirm a recent recoverable backup/restore point according to the Neon plan and team policy. Backup/restore procedure is not documented in this repository.
- Apply only the migrations required to bring the target forward, in numbered order, using the organization's approved SQL access path. Do not assume files are safe to rerun unless the migration README states that.
- Migration 005 alters weight_entries precision and adds a range constraint; verify existing data satisfies the supported range before applying in a divergent environment.
- After applying: verify the expected tables, columns, constraints, indexes and foreign keys; run application-level smoke checks against the same target.
- Record migration filenames/versions, target branch, timestamp, operator and outcome. Never paste DATABASE_URL into logs or handover records.

Migration rollback posture

No general rollback tooling or reverse SQL scripts were verified. For additive migrations, preferred recovery may be a forward fix or database restore, depending on the data change and environment. Do not drop tables or reverse a migration in production without an approved recovery plan and confirmed backup.

4. Post-release smoke checks

Area	Check	Success signal
Application availability	Open app home and Streamlit navigation.	Page loads; no setup exception.
Database connection	Read a known profile or page listing using intended environment.	Current data returns without exposing credentials.
Profile/check-in	Read profile; on approved test account/branch, save and reload a check-in.	Values persist and date uniqueness/upsert behavior is correct.
Recipe	Open library; validate CSV preview without committing unintended changes.	Expected rows/validation messages appear.
Meal plan	Read selected date; use test plan date only if write testing authorized.	Date-specific plan loads; no cross-date mutation.
Voice	Check manual text fallback; test mic only with permission and non-sensitive text.	Reviewed text is editable; failure still leaves text path available.

Use a staging/test Neon branch for write checks. This runbook does not authorize test writes against production data.

5. Monitoring and support

The repository exposes user-readable errors for missing DATABASE_URL and catches database/value errors in page flows. A centralized logging, metrics, alerting or incident-management integration was not identified in the reviewed files. Confirm a monitoring owner and provider before production support commitments.

- Capture error type, operation and timestamp without logging secrets or sensitive meal/profile content.
- Separate application errors from database connectivity, missing migration, browser microphone and third-party speech recognition issues.
- When a user reports a failed save, confirm whether the transaction committed before asking them to repeat it; date uniqueness/upserts reduce some duplicates, but not every action is idempotent.
- Document where hosting logs are available, who can access them, and retention periods.

6. Common failure response

Symptom	Likely check	Response
Missing DATABASE_URL	Environment variable or Streamlit secret configured for this app/environment?	Set protected secret; restart/redeploy as host requires; retry read.
Relation/column does not exist	Was required migration applied to the exact Neon branch?	Pause dependent release; apply reviewed migration then smoke-check.
Connection refused/timeout	Neon endpoint, branch status, network allow-list, host egress and service status.	Confirm target and connectivity; retry after recovery; do not switch to another DB silently.
Constraint rejection	Submitted value and schema/code version alignment.	Preserve user input; correct data or code/schema mismatch; no success message.
Speech control unavailable	Browser support, microphone permission, HTTPS and browser speech privacy.	Use manual text entry; plan page remains independently accessible.
CSV import failed	Both files, exact headers, values, references, transaction error.	Review row errors and preview; retry only after confirming no partial commit.

7. Rollback and recovery decisions

Application rollback and database rollback are separate actions. The repository does not establish an automated deployment target or a rollback command, so these steps need environment-specific confirmation before a production incident.

- If a release breaks the UI but schema remains backward-compatible, redeploy the last known-good application revision through the chosen host's documented process. Verify current hosting semantics first.
- If a release depends on a newly applied schema, reverting code may not reverse the schema change. Prefer forward correction or restore to a known point based on data impact and the approved recovery objective.
- If records may have been partially changed, inspect transaction boundaries and affected rows before retrying. Repository connection context rolls back exceptions within a transaction; multi-request UI flows may still have completed earlier independent transactions.
- Confirm backup frequency, restore-point availability, recovery-point objective (RPO), recovery-time objective (RTO), and who can authorize restoration. These are currently undocumented.

8. Operational ownership and release record

Before production release, assign named owners for the application, Neon database, hosting configuration, CI workflow, secrets, and incident response. No names, schedules or service levels were discoverable in the repository.

Release record field	Capture for each release
Identity	Version/commit SHA, branch, release time, operator, change summary.
Validation	CI run for SHA, migration list, smoke-check results and known limitations.
Environment	Hosting target and Neon branch identifier (not credential).
Recovery	Last known-good app version; restore-point/backup confirmation; rollback owner.
Follow-up	Open incidents, deferred changes and next owner/action.

9. Release acceptance criteria

OPS-01 Candidate commit has a completed passing CI run using the repository workflow, or a documented approved exception.

OPS-02 DATABASE_URL is configured in a protected secret store for the correct environment and is not present in source or release logs.

OPS-03 Required migrations are applied to the intended Neon branch and recorded before dependent code is released.

OPS-04 Application starts and critical pages complete a read smoke check against the target environment.

OPS-05 Any write smoke check uses a staging/test branch or explicitly approved test record; production write test is not assumed.

OPS-06 Release record captures SHA, CI status, migration state, hosting target, smoke checks and recovery contact.

OPS-07 Future A documented backup/restore and app rollback procedure has owners and has been exercised within the agreed recovery objectives.

OPS-08 Future Monitoring and alert routing identify application, database, migration and speech-recognition failure categories without exposing sensitive data.

10. Evidence and limits

Reviewed current GitHub main files: README.md, .env.example, requirements.txt, .gitignore, database/migrations/README.md, docs/database/schema.md, bulking_app/neon.py, app.py, and .github/workflows/ci.yml. Lookup of .github/workflows/deploy.yml returned not found. Local project mirror was also reviewed.

The deployment platform, production URL, active Neon branch, secret values, CI run status, applied migration state, backup configuration, monitoring and rollback method were not inspected or verified. No tests or deployment actions were run. The PDF was text-checked and visually rendered.